The Entity-Relationship Model - Toward a Unified View of Data

By Peter Chen

Presented by Prof. Shahram Ghandeharizadeh, CSCI 585, 8/24/2026, USC

Assigned Reading



The Entity-Relationship Model—Toward a Unified View of Data

PETER PIN-SHAN CHEN

Massachusetts Institute of Technology

A data model, called the entity-relationship model, is proposed. This model incorporates some of the important semantic information about the real world. A special diagrammatic technique is introduced as a tool for database design. An example of database design and description using the model and the diagrammatic technique is given. Some implications for data integrity, information retrieval, and data manipulation are discussed.

The entity-relationship model can be used as a basis for unification of different views of data: the network model, the relational model, and the entity set model. Semantic ambiguities in these models are analyzed. Possible ways to derive their views of data from the entity-relationship model are presented.

Key Words and Phrases: database design, logical view of data, semantics of data, data models, entity-relationship model, relational model, Data Base Task Group, network model, entity set model, data definition and manipulation, data integrity and consistency
CR Categories: 3.50, 3.70, 4.33, 4.34

1. INTRODUCTION

The logical view of data has been an important issue in recent years. Three major data models have been proposed: the network model [2, 3, 7], the relational model [8], and the entity set model [25]. These models have their own strengths and weaknesses. The network model provides a more natural view of data by separating entities and relationships (to a certain extent), but its capability to achieve data independence has been challenged [8]. The relational model is based on relational

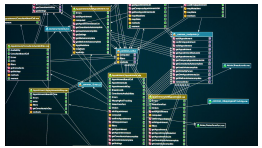
Entity-Relationship (E-R) Model

Level 1



Conceptual data model: Entities, Relationships, Values.

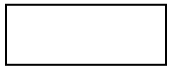
Level 2



Representation of conceptual objects:

- How to represent entities and relationships?
  - Tables of rows and columns.
- Keys

Entity-Relationship (E-R) MODEL

- E-R model is based on a real world that consists of entities and relationships. It consists of two primitives:
  1. **Entities**: objects that exist and are distinguishable from one another. These objects might be abstract, e.g., SF-49ers, Joe, Bob, etc.
  2. **Relationships**: two or more entities may participate in a relationship, e.g., Joe is a fan of SF-49ers, etc.
- An entity is represented by a set of *attributes,* e.g., Joe might have a social-security number, a phone number, a spouse, and a salary 
- An ***attribute*** is a function that maps an entity into a domain, e.g., Joe's social-security number maps Joe into the domain of integers. An attribute is represented as an 
- A set of entities of the same type is termed an ***entity set***, e.g., football teams. Entity sets do not need to be disjoint, e.g., students, employees, people. An entity set is represented as a

Entity-Relationship (E-R) MODEL

- E-R model is based on a real world that consists of entities and relationships. It consists of two primitives:
  1. **Entities**: objects that exist and are distinguishable from one another. These objects might be abstract, e.g., SF-49ers, Joe, Bob, etc.
  2. **Relationships**: two or more entities may participate in a relationship, e.g., Joe is a fan of SF-49ers, etc.

and DEPARTMENT. There is a predicate associated with each entity set to test whether an entity belongs to it. For example, if we know an entity is in the entity set EMPLOYEE, then we know that it has the properties common to the other entities in the entity set EMPLOYEE. Among these properties is the aforementioned test predicate. Let E_i denote entity sets. Note that entity sets may not

- A set of entities of the same type is termed an ***entity set***, e.g., football teams. Entity sets do not need to be disjoint, e.g., students, employees, people. An entity set is represented as a

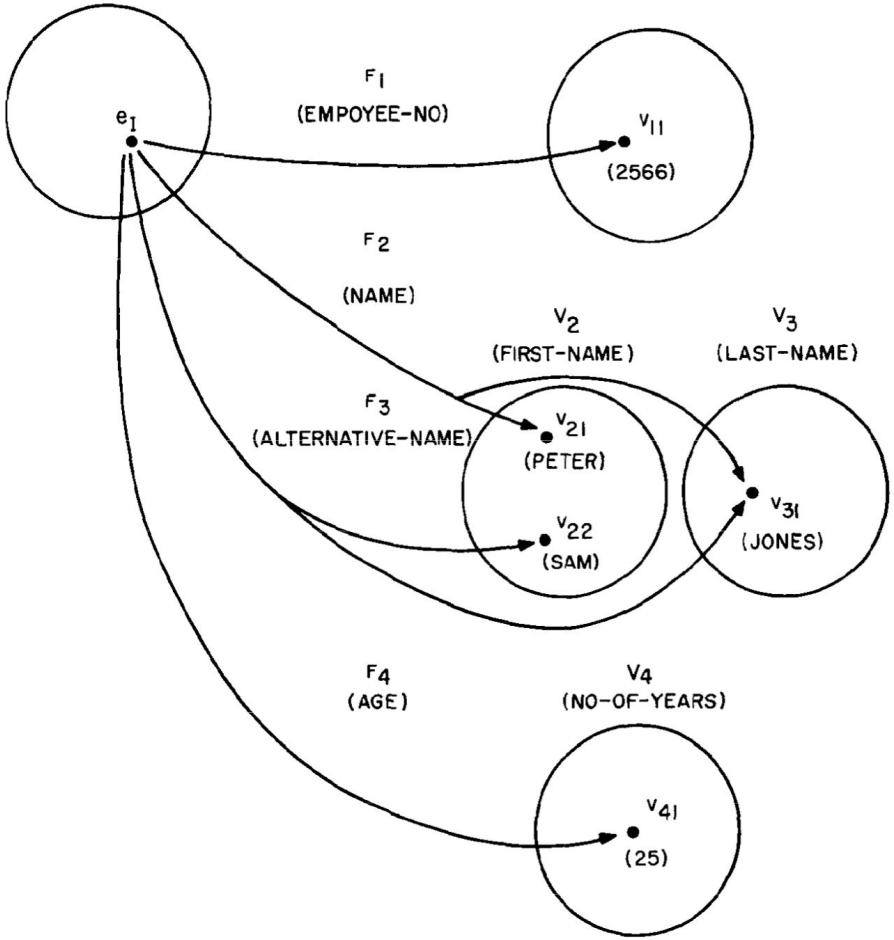


Fig. 2. Attributes defined on the entity set PERSON

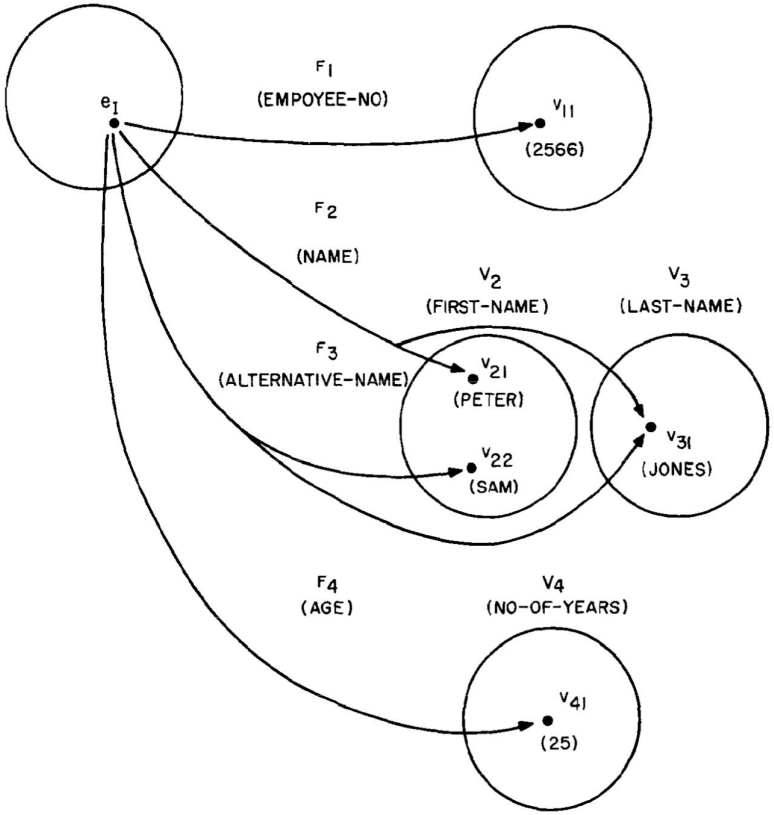
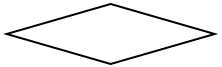


Fig. 2. Attributes defined on the entity set PERSON

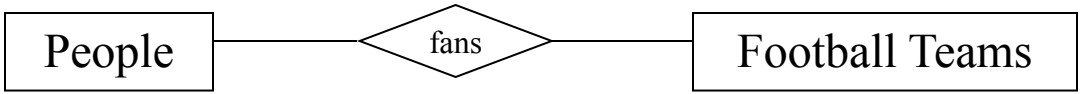
Figure 2 illustrates some attributes defined on entity set PERSON. The attribute AGE maps into value set NO-OF-YEARS. An attribute can map into a Cartesian product of value sets. For example, the attribute NAME maps into value sets FIRST-NAME, and LAST-NAME. Note that more than one attribute may map from the same entity set into the same value set (or same group of value sets). For example, NAME and ALTERNATIVE-NAME map from the entity set EMPLOYEE into value sets FIRST-NAME and LAST-NAME. Therefore, attribute and value set are different concepts although they may have the same name in some cases (for example, EMPLOYEE-NO maps from EMPLOYEE to value set EMPLOYEE-NO). This distinction is not clear in the network model and in many existing data management systems. Also note that an attribute is defined as a function. Therefore, it maps a given entity to a single value (or a single tuple of values in the case of a Cartesian product of value sets).

RELATIONSHIPS

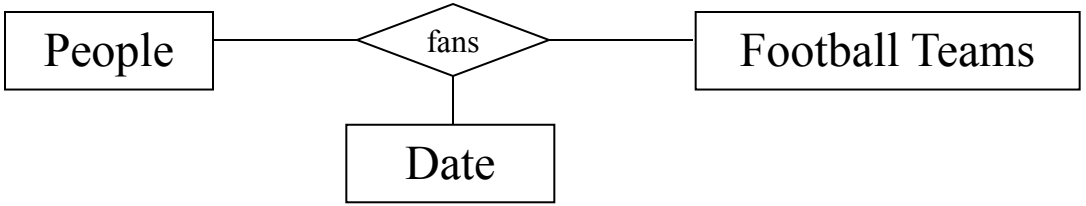
- Relationships of the same type are contained in a relationship set. More formally, if $E_1, E_2, ..., E_n$ are entity sets, then a relationship set R is a subset of: $\{[e_1, e_2, ..., e_n] \mid e_1 \in E_1, e_2 \in E_2, ..., e_n \in E_n\}$ where $[e_1, e_2, ..., e_n]$ is a relationship. A relationship set is represented as a



- A relationship may involve two entities, binary relationship:



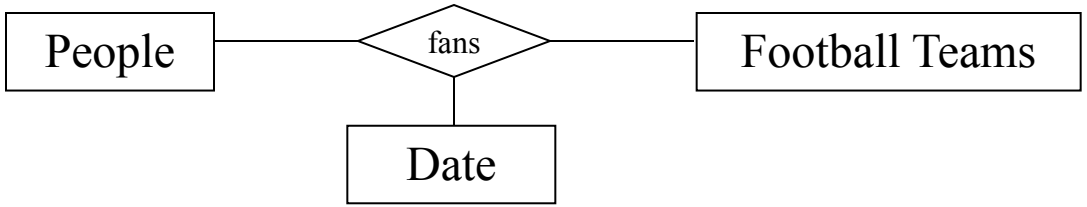
- A relationship may involve three entities, ternary relationship:



RELATIONSHIPS

- Relationships of the same type are contained in a relationship set. More formally, if $E_1, E_2, ..., E_n$ are entity sets, then a relationship set R is a subset of: $\{[e_1, e_2, ..., e_n] \mid e_1 \in E_1, e_2 \in E_2, ..., e_n \in E_n\}$ where $[e_1, e_2, ..., e_n]$ is a relationship. A relationship set is represented as a

The *role* of an entity in a relationship is the function that it performs in the relationship. "Husband" and "wife" are roles. The ordering of entities in the definition of relationship (note that square brackets were used) can be dropped if roles of entities in the relationship are explicitly stated as follows: $(r_1/e_1, r_2/e_2, \ldots, r_n/e_n)$, where r_i is the role of e_i in the relationship.

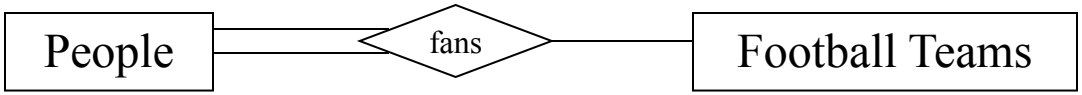


*

RELATIONSHIPS

- A relationship does not require participation of entities from a set. In the previous example, John as an entity in "People" entity-set and is not required to participate in a relationship with "Football Teams" entity-set.

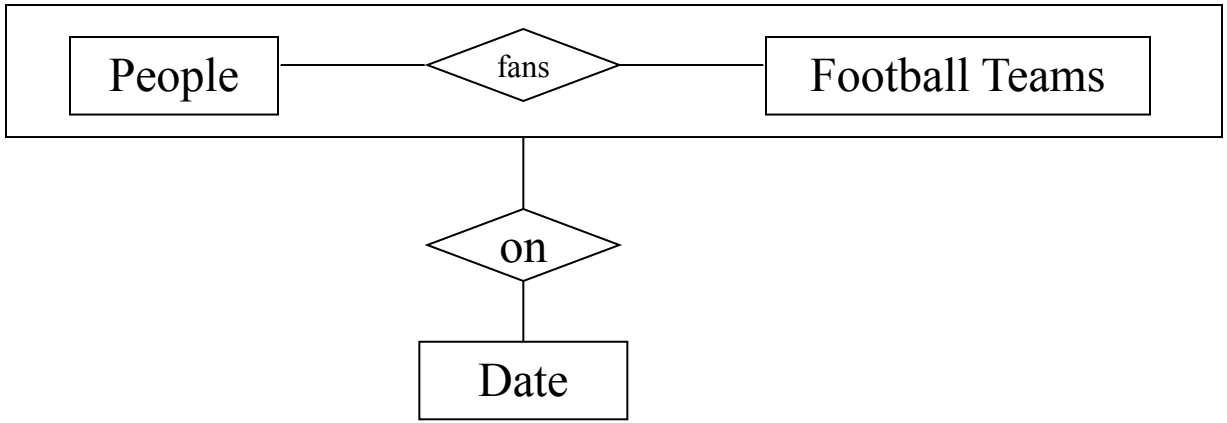
- There are times, when one may want to specify that all members of an entity set MUST participate in a relationship. Use double lines to represent this case.



- The double lines specifies ***total participation*** of entities in People entity set in the "fans" relationship set.

RELATIONSHIPS (Cont…)

- A relationship may involve *n* entities, N-ary relationship
- It is always possible to replace a non-binary relationship set by a number of distinct binary relationship sets



*

RELATIONSHIPS (Cont…)

- Example: A library database contains a listing of authors that have written books on various subjects (one author per book). It also contains information about libraries that carry books on various subjects.

RELATIONSHIPS (Cont…)

- Example: A library database contains a listing of authors that have written books on various subjects (one author per book). It also contains information about libraries that carry books on various subjects.

There are four steps in designing a database using the entity-relationship model: (1) identify the entity sets and the relationship sets of interest; (2) identify semantic information in the relationship sets such as whether a certain relationship

[2] This mapping information is included in DIAM II [24].

ACM Transactions on Database Systems, Vol. 1, No. 1, March 1976.

set is an 1:n mapping; (3) define the value sets and attributes; (4) organize data into entity/relationship relations and decide primary keys.

*

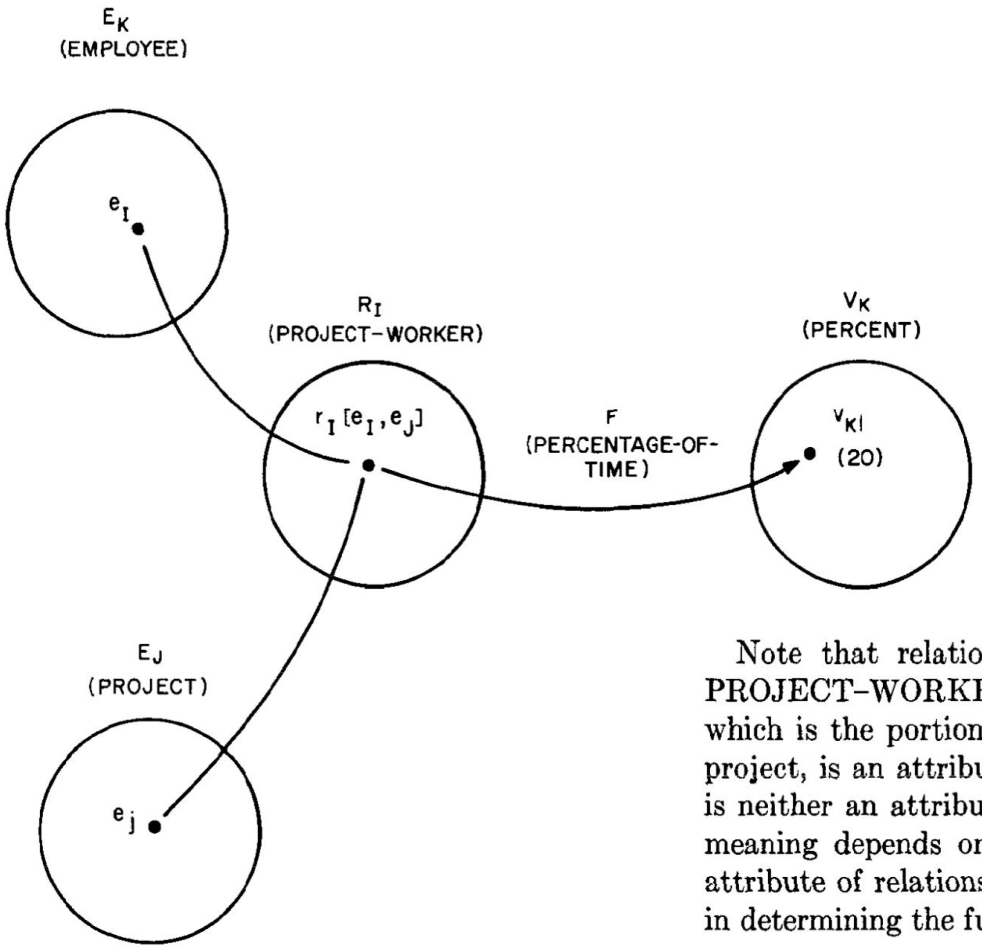
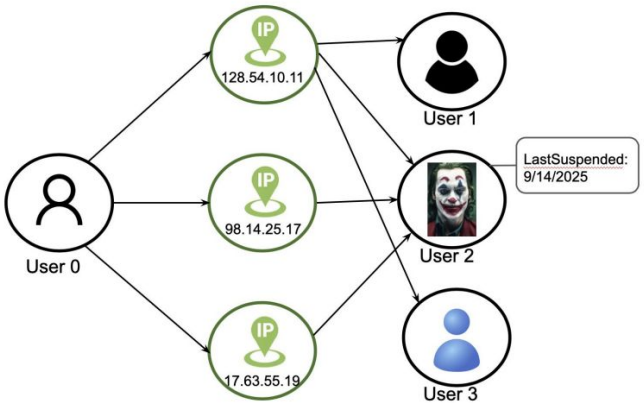


Fig. 3. Attributes defined on the relationship set PROJECT-WORKER

Note that relationships also have attributes. Consider the relationship set PROJECT-WORKER (Figure 3). The attribute PERCENTAGE-OF-TIME, which is the portion of time a particular employee is committed to a particular project, is an attribute defined on the relationship set PROJECT-WORKER. It is neither an attribute of EMPLOYEE nor an attribute of PROJECT, since its meaning depends on both the employee and project involved. The concept of attribute of relationship is important in understanding the semantics of data and in determining the functional dependencies among data.

Fraud Detection

Given a user, say User 0, check whether the user is suspicious via IP address linking.

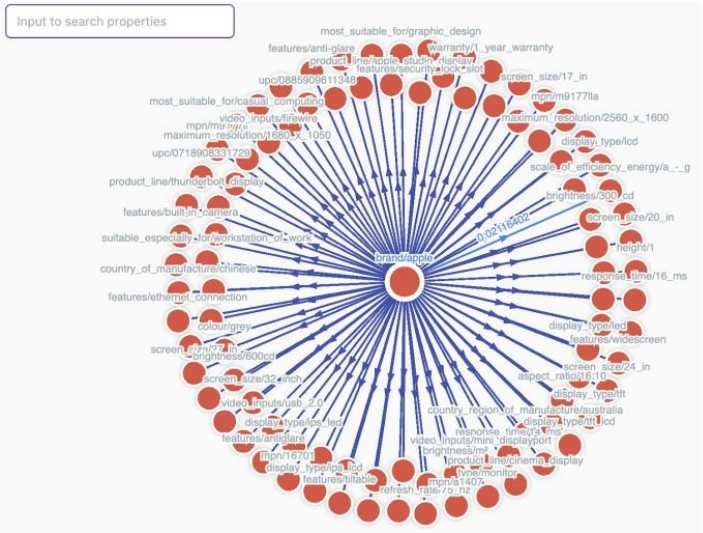


Observations:

1. This interactive query is user facing and requires fast response time.
2. A query may use other attributes such as Device, Payment, Phone, Gov ID, etc.

Product Recommendation

When a user views a specific item, say an iPhone, the query generates a list of alternative models (e.g., Samsung Galaxy or Google Pixel) as personalized recommendations.

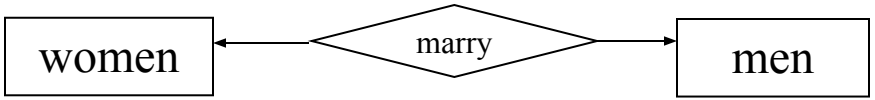


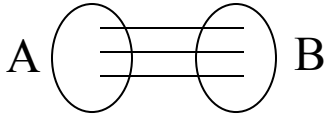
Observation: Observations 1 and 2 of previous slide + Hundreds of millions of entities and relationships.

BINARY RELATIONSHIP

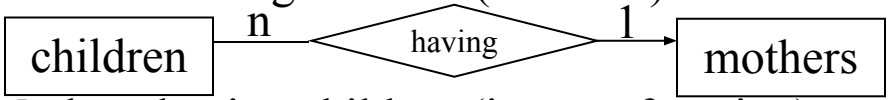
A binary relationship between entity set *A* and *B* might be:

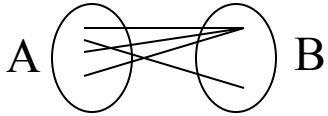
- 1:1 Women marrying Men (function)



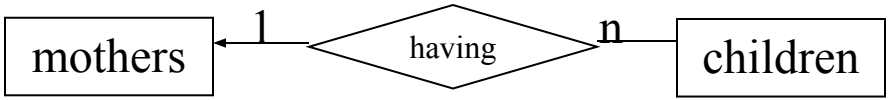


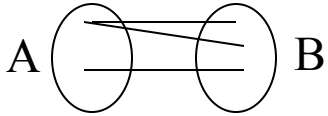
- N:1 Children having mothers (function)



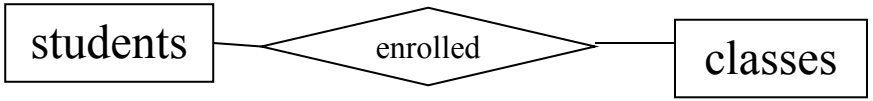


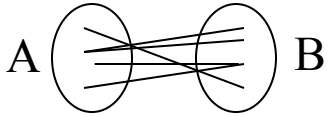
- 1:N Mothers having children (inverse function)





- M:N Students enrolled in a class





Information Structure (Level 2): KEY

- Entities and relationships are distinguishable using various keys
- A ***key*** is a combination of one or more attributes, e.g., social-security number, combination of name and social-security number.

2.3 Information Structure (Level 2)

The entities, relationships, and values at level 1 (see Figures 2–5) are conceptua
objects in our minds (i.e. we were in the conceptual realm [18, 27]). At level 2
we consider representations of conceptual objects. We assume that there exist
direct representations of values. In the following, we shall describe how to represent
entities and relationships.

- A ***primary key*** is a candidate key that is chosen by the database designer to identify the entities of an entity set.

Information Structure (Level 2): KEY

- Entities and relationships are distinguishable using various keys
- A ***key*** is a combination of one or more attributes, e.g., social-security number, combination of name and social-security number.

- A ***superkey*** is a key defined either for an entity set or relationship set that uniquely identifies an entity, e.g., social-security number, phone number, combination of name and social-security number.

- A ***candidate key*** is a minimal superkey that uniquely identifies either an entity or a relationship, e.g., social-security number, phone number.

- A ***primary key*** is a candidate key that is chosen by the database designer to identify the entities of an entity set. It is a *semantically meaningful* key.

Information Structure (Level 2): KEY

Table I. Insertion

level 1	level 2
operation: insert an entity to an entity set	*operation:* create an entity tuple with a certain entity-PK *check:* whether PK already exists or is acceptable
operation: insert a relationship in a relationship set *check:* whether the entities exist	*operation:* create a relationship tuple with given entity PKs *check:* whether the entity PKs exist
operation: insert properties of an entity or a relationship *check:* whether the value is acceptable	*operation:* insert values in an entity tuple or a relationship tuple *check:* whether the values are acceptable

Information Structure (Level 2): KEY

Table II. Updating

level 1	level 2
operation: • change the value of an entity attribute	*operation:* • update a value *consequence:* • if it is not part of an entity PK, no consequence • if it is part of an entity PK, •• change the entity PKs in all related relationship relations •• change PKs of other entities which use this value as part of their PKs (for example, DEPENDENTS' PKs use EMPLOYEE'S PK)
operation: • change the value of a relationship attribute	*operation:* • update a value (note that a relationship attribute will not be a relationship PK)

Information Structure (Level 2): KEY

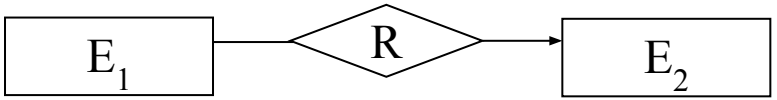
Table III. Deletion

level 1	level 2
operation: • delete an entity *consequences:* • delete any entity whose existence depends on this entity • delete relationships involving this entity • delete all related properties	*operation:* • delete an entity tuple *consequences* (applied recursively): • delete any entity tuple whose existence depends on this entity tuple • delete relationship tuples associated with this entity
operation: • delete a relationship *consequences:* • delete all related properties	*operation:* • delete a relationship tuple

KEY (Cont…)

- Relationship sets also have primary keys. Assume R is a relationship set involving entity sets $E_1, E_2, ..., E_n$. Let primary-key(E_i) denote the primary key for entity set E_i. Assume primary-key(E_i) is unique for $1 \leq i \leq n$. If R has no attributes then its superkey is:

  $primary\text{-}key(E_1) \cup primary\text{-}key(E_2) \cup ... \cup primary\text{-}key(E_n)$
- This is a primary key if the mapping constraint is many-to-many.
- If the mapping constraint is many to one from E_1 to E_2 then the primary key of R is primary key of E_1.



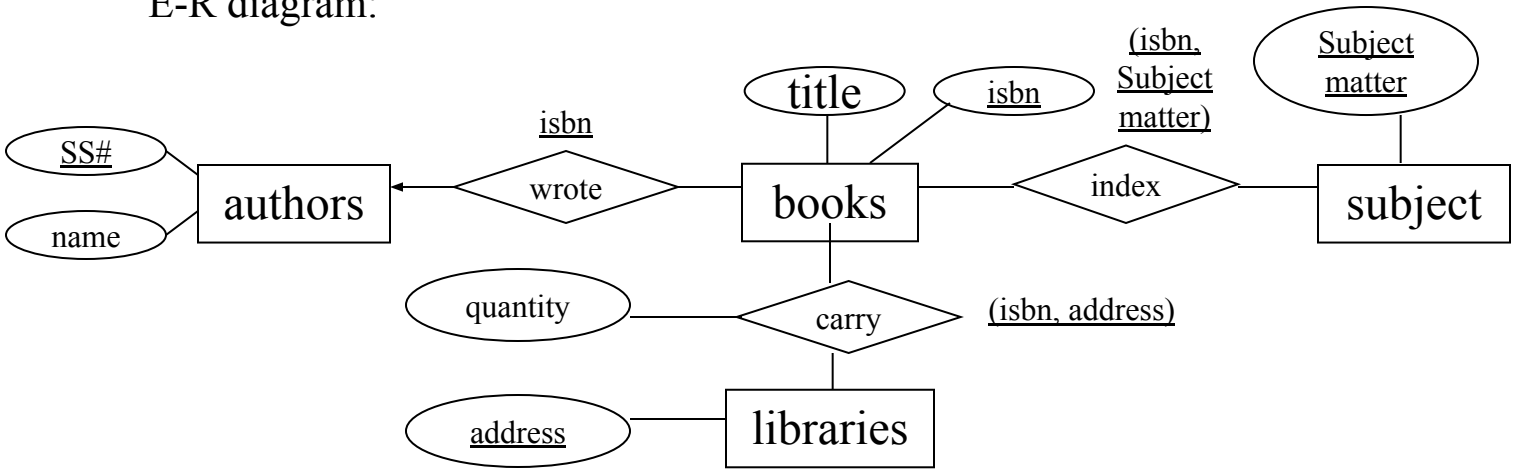
RELATIONSHIPS (Cont…)

- Example: A library database contains a listing of authors that have written books on various subjects (one author per book). It also contains information about libraries that carry books on various subjects.

  Entity sets: authors, subjects, books, libraries

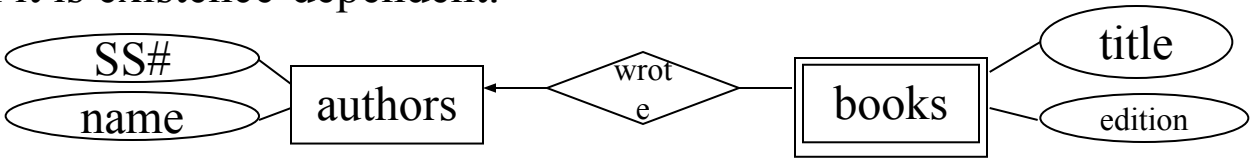
  Relationship sets: wrote, carry, indexed

  E-R diagram:



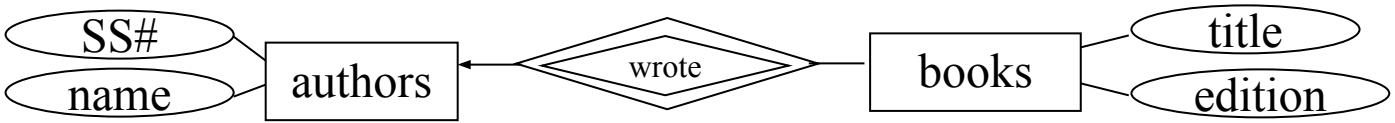
KEY (Cont…)

- A ***foreign key*** is a set of one or more attributes of a strong entity set that are employed to construct the discriminator of a weak entity set. The primary key of a weak entity set is formed by the primary key of the strong entity set on which it is existence-dependent.



Weak and Strong Entity Sets

- An entity set that does not have sufficient attributes to form a primary key is termed a ***weak entity set.*** An entity set that has a primary key is termed a ***strong/regular entity set.***
- A weak entity set must be associated with an ***identifying*** or ***owner entity set.***
  - Every weak entity must be associated with an ***identifying entity.*** Its ***existence depends*** on the identifying entity.
  - The identifying entity is said to ***own*** the weak entity set that it identifies.
  - The relationship associating the weak entity set with the identifying entity set is called the ***identifying relationship.***
- The identifying relationship is one-to-one or many-to-one from the weak entity set to the identifying entity set AND the participation of the weak entity set in the relationship is total.



EXAMPLE

- Employees of a large company, e.g., IBM, where an employee reports to a manager. The manager is also an employee who reports to another manager. This chain of command continues to the very top where the CEO is the only employee who is not reporting to a manager. Draw the ER diagram for this example.

Manufacturing Company of Section 3.1

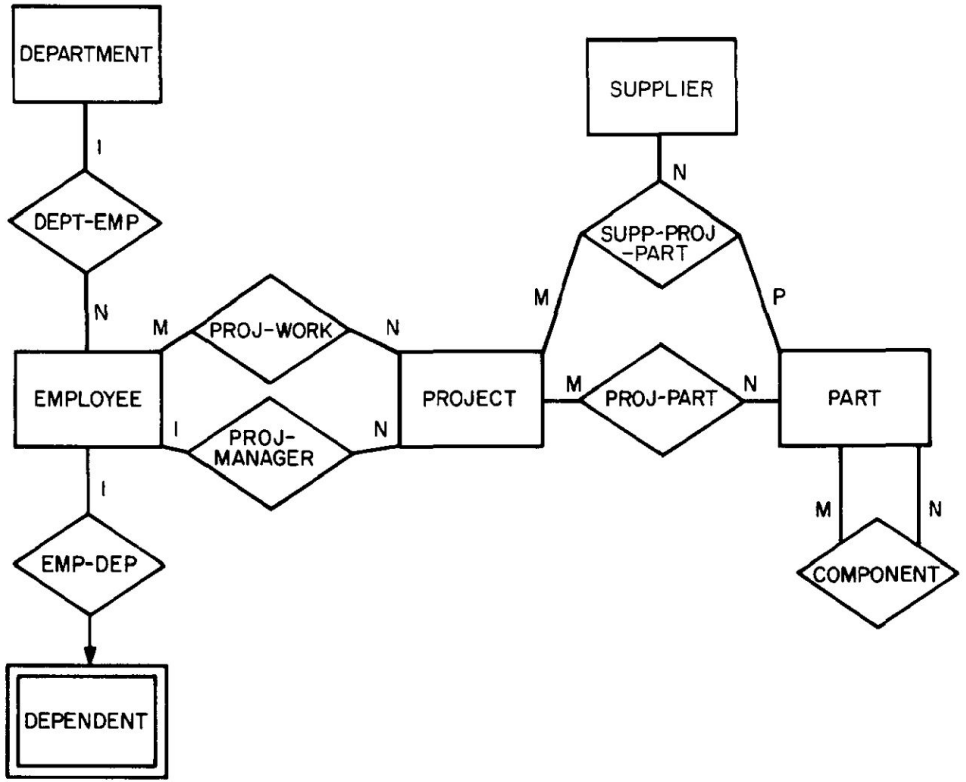


Fig. 11. An entity-relationship diagram for analysis of information in a manufacturing firm

- Relationship sets defined on more than two entity sets.
- A relationship set defined on only one entity set.
- More than one relationship set defined on a given entity set.
- Mapping cardinalities: 1:1, 1:n, n:m.
- Existence dependency of one entity on another.

Reduction to Tables (Relations)

DECLARE VALUE-SETS REPRESENTATION ALLOWABLE-VALUES

EMPLOYEE-NO	INTEGER (4)	(0,2000)
FIRST-NAME	CHARACTER (8)	ALL
LAST-NAME	CHARACTER (10)	ALL
NO-OF-YEARS	INTEGER (3)	(0,100)
PROJECT-NO	INTEGER (3)	(1,500)
PERCENTAGE	FIXED (5.2)	(0,100.00)

DECLARE REGULAR ENTITY RELATION EMPLOYEE
ATTRIBUTE/VALUE-SET:
    EMPLOYEE-NO/EMPLOYEE-NO
    NAME/(FIRST-NAME, LAST-NAME)
    ALTERNATIVE-NAME/(FIRST-NAME,LAST-NAME)
    AGE/NO-OF-YEARS
PRIMARY KEY:
    EMPLOYEE-NO

DECLARE REGULAR ENTITY RELATION PROJECT
ATTRIBUTE/VALUE-SET:
    PROJECT-NO/PROJECT-NO
PRIMARY KEY:
    PROJECT-NO

DECLARE REGULAR RELATIONSHIP RELATION PROJECT-WORKER
ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
    WORKER/EMPLOYEE.PK/m
    PROJECT/PROJECT.PK/n (m:n mapping)
ATTRIBUTE/VALUE-SET:
    PERCENTAGE-OF-TIME/PERCENTAGE

DECLARE WEAK RELATIONSHIP RELATION EMPLOYEE-DEPENDENT
ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
    SUPPORTER/EMPLOYEE.PK/1
    DEPENDENT/DEPENDENT.PK/n
EXISTENCE OF DEPENDENT DEPENDS ON
EXISTENCE OF SUPPORTER

DECLARE WEAK ENTITY RELATION DEPENDENT
ATTRIBUTE/VALUE-SET:
    NAME/FIRST-NAME
    AGE/NO-OF-YEARS
PRIMARY KEY:
    NAME
    EMPLOYEE.PK THROUGH EMPLOYEE-DEPENDENT

Tables/Relations: Data Integrity Constraints (Allowable Values)

DECLARE VALUE-SETS REPRESENTATION ALLOWABLE-VALUES

EMPLOYEE-NO	INTEGER (4)	(0,2000)
FIRST-NAME	CHARACTER (8)	ALL
LAST-NAME	CHARACTER (10)	ALL
NO-OF-YEARS	INTEGER (3)	(0,100)
PROJECT-NO	INTEGER (3)	(1,500)
PERCENTAGE	FIXED (5.2)	(0,100.00)

DECLARE REGULAR ENTITY RELATION EMPLOYEE
ATTRIBUTE/VALUE-SET:
EMPLOYEE-NO/EMPLOYEE-NO
NAME/(FIRST-NAME, LAST-NAME)
ALTERNATIVE-NAME/(FIRST-NAME,LAST-NAME)
AGE/NO-OF-YEARS
PRIMARY KEY:
EMPLOYEE-NO

DECLARE REGULAR ENTITY RELATION PROJECT
ATTRIBUTE/VALUE-SET:
PROJECT-NO/PROJECT-NO
PRIMARY KEY:
PROJECT-NO

DECLARE REGULAR RELATIONSHIP RELATION PROJECT-WORKER
ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
WORKER/EMPLOYEE.PK/m
PROJECT/PROJECT.PK/n (m:n mapping)
ATTRIBUTE/VALUE-SET:
PERCENTAGE-OF-TIME/PERCENTAGE

DECLARE WEAK RELATIONSHIP RELATION EMPLOYEE-DEPENDENT
ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
SUPPORTER/EMPLOYEE.PK/1
DEPENDENT/DEPENDENT.PK/n
EXISTENCE OF DEPENDENT DEPENDS ON
EXISTENCE OF SUPPORTER

DECLARE WEAK ENTITY RELATION DEPENDENT
ATTRIBUTE/VALUE-SET:
NAME/FIRST-NAME
AGE/NO-OF-YEARS
PRIMARY KEY:
NAME
EMPLOYEE.PK THROUGH EMPLOYEE-DEPENDENT

Constraint on allowable values.

$$\text{AGE}(e) \in (20,65), \text{ where } e \in \text{EMPLOYEE}$$

Tables/Relations: Data Integrity Constraints (Existing Values)

DECLARE	VALUE-SETS	REPRESENTATION	ALLOWABLE-VALUES
	EMPLOYEE-NO	INTEGER (4)	(0,2000)
	FIRST-NAME	CHARACTER (8)	ALL
	LAST-NAME	CHARACTER (10)	ALL
	NO-OF-YEARS	INTEGER (3)	(0,100)
	PROJECT-NO	INTEGER (3)	(1,500)
	PERCENTAGE	FIXED (5.2)	(0,100.00)

DECLARE REGULAR ENTITY RELATION EMPLOYEE
ATTRIBUTE/VALUE-SET:
EMPLOYEE-NO/EMPLOYEE-NO
NAME/(FIRST-NAME, LAST-NAME)
ALTERNATIVE-NAME/(FIRST-NAME,LAST-NAME)
AGE/NO-OF-YEARS
PRIMARY KEY:
EMPLOYEE-NO

(3) Constraints on *existing values* in the database. There are two types of constraints:

(i) Constraints between sets of existing values. For example,

$$\{\text{NAME}(e) \mid e \in \text{MALE-PERSON}\} \subseteq \{\text{NAME}(e) \mid e \in \text{PERSON}\}.$$

(ii) Constraints between particular values. For example,

$$\text{TAX}(e) \leq \text{SALARY}(e),\ e \in \text{EMPLOYEE}$$

or

$$\text{BUDGET}(e_i) = \sum \text{BUDGET}(e_j), \text{ where } e_i \in \text{COMPANY}, \ e_j \in \text{DEPARTMENT}$$
$$\text{and } [e_i, e_j] \in \text{COMPANY-DEPARTMENT}.$$

DECLARE REGULAR ENTITY RELATION PROJECT
ATTRIBUTE/VALUE-SET:
PROJECT-NO/PROJECT-NO
PRIMARY KEY:
PROJECT-NO

DECLARE REGULAR RELATIONSHIP RELATION PROJECT-WORKER
ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
WORKER/EMPLOYEE.PK/m
PROJECT/PROJECT.PK/n (m:n mapping)
ATTRIBUTE/VALUE-SET:
PERCENTAGE-OF-TIME/PERCENTAGE

DECLARE WEAK RELATIONSHIP RELATION EMPLOYEE-DEPENDENT
ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
SUPPORTER/EMPLOYEE.PK/1
DEPENDENT/DEPENDENT.PK/n
EXISTENCE OF DEPENDENT DEPENDS ON
EXISTENCE OF SUPPORTER

DECLARE WEAK ENTITY RELATION DEPENDENT
ATTRIBUTE/VALUE-SET:
NAME/FIRST-NAME
AGE/NO-OF-YEARS
PRIMARY KEY:
NAME
EMPLOYEE.PK THROUGH EMPLOYEE-DEPENDENT

Entity-Relationship Model: Functional Dependency

Level 1



Conceptual data model: Entities, Relationships, Values.
Functional dependency: An attribute is a function. It maps an entity in an entity set to a single value in value set.

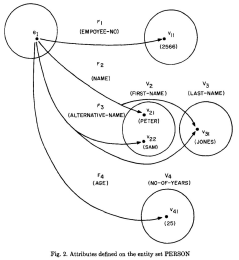
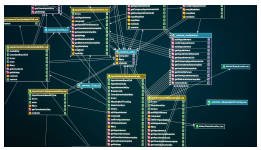


Fig. 2. Attributes defined on the entity set PERSON

Level 2



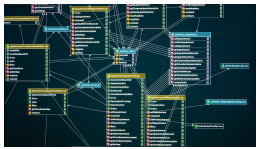
Representation of conceptual objects:

- How to represent entities and relationships?
  - Tables of rows and columns.
- Keys

Functional dependency: The values of the primary key are used to represent entities. Therefore, nonkey value sets (domains) are functionally dependent on primary-key value sets.

Level 2: Functional Dependency

Level 2



- Functional dependency: An attribute *Y* is *functionally dependent* on attribute (or set of attributes) *X* if it is invalid to have two records with the same *X* value but different *Y* values. A given *X* value must always occur with the same *Y* value.

- If *X* is the key of a relation then all other attributes of a relation are by definition functionally dependent on *X*.

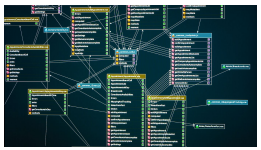
Multi-Valued Attributes

Level 1



Conceptual data model: Entities, Relationships, Values.
Multi-valued attributes are allowed.

Level 2



Representation of conceptual objects:

- How to represent entities and relationships?
  - Tables of rows and columns.
- Keys

Multi-valued attributes are <u>not</u> allowed. Violates 1st normal form. Why?

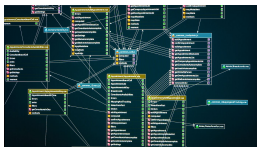
2nd & 3rd Normal Forms

Level 1



Conceptual data model: Entities, Relationships, Values.
Top-down approach: 2nd and 3rd Normal Forms are not applicable. Why?

Level 2



Representation of conceptual objects:

- How to represent entities and relationships?
  - Tables of rows and columns.
- Keys

2nd and 3rd Normal Forms apply.

Level 2: Violating 2nd Normal Form

- The second normal form is violated when a non-key field is a fact about a subset of a key. This circumstance arises when the key is composite.

PART	WAREHOUSE	QUANTITY	WAREHOUSE-ADDRESS

……key……

- Limitations:
  1. Warehouse address is repeated in every record (every part)
  2. If the address of a warehouse changes, many records must be updated
  3. The data may become inconsistent if the update is not done correctly
  4. If a warehouse becomes empty, the database management system might lose track of it because no records may indicate that the warehouse exists in the database.

Level 2, Violating 2nd Normal Form: Solution

- Solution: Satisfy the second normal form by decomposing the original relation into two different relations

PART	WAREHOUSE	QUANTITY

.....key.....

WAREHOUSE	WAREHOUSE-ADDRESS

.....key.....

This concept does NOT apply to Level 1.

Normalization

- Definition: The process of replacing un-normalized records with normalized records is termed normalization.
- Advantage of normalization: it enhances the integrity of data by minimizing redundancy and inconsistency.
- Disadvantage of normalization: performance might be lost. For example, the execution of the following query now requires the execution of a join operator: Retrieve the address of all warehouses that contain part-id=5.

Level 2: Violating 3rd Normal Form

- The third normal form is violated when a non-key attribute is a fact about another non-key attribute:

EMPLOYEE	DEPARTMENT	LOCATION

….key……

- Address is a fact about department and not the faculty. In addition to suffering from the limitations of the second normal form, violating the third normal form suffers from update ambiguities: Logically speaking, an update that changes address of a faculty (say Shahram) from SAL to HNB has two alternative meaning: either
  1. Shahram is moving to a different department and his department attribute value will be updated later or
  2. the computer science department is changing address and all the faculty in this department will observe an update in their address attribute later.
- It is difficult to design algorithms to maintain consistency with such ambiguities.

Level 2, Violating 3nd Normal Form: Solution (Normalization

- Solution: decompose the violating record into two.



This concept does NOT apply to Level 1.

Network Model

- Network model is the semantics of a data structure diagram. Each of its rectangular box represents a record type.
- The arrow represents a data-structure-set in which the DEPT record is the owner-record, and one owner-record may own n (n=0,1,2,...) member records.

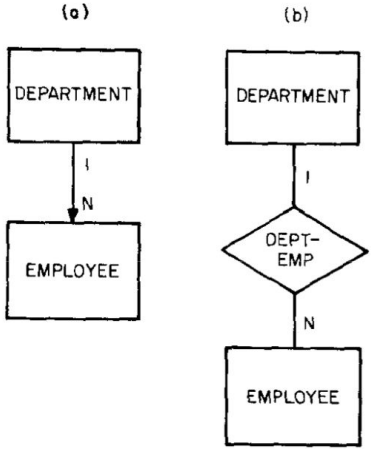


Fig. 16. Relationship DEPARTMENT–EMPLOYEE
(a) data structure diagram
(b) entity-relationship diagram

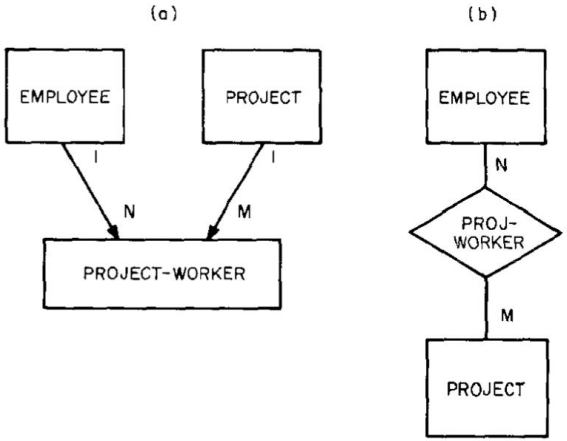
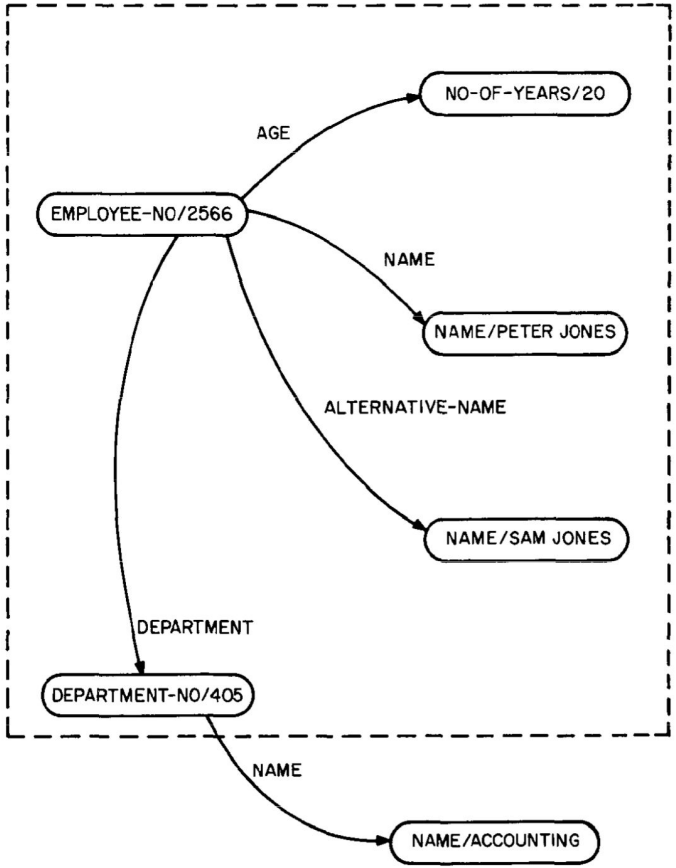


Fig. 17. Relationship PROJECT–WORKER
(a) data structure diagram
(b) entity–relationship diagram

Entity Set Model

- The basic element of the entity set model is the entity. Entities have names (entity names) such a "Peter Jones", "blue", or "22". Entity names having some properties in common are collected into an entity-name-set which is referenced by the entity-name-set-name such as "NAME", "COLOR", and "QTY".
- An entity is represented by entity-name-set-name/entity-name pair such as NAME/Peter Joes, QTY/22.
- An entity is described by its association with other entities.

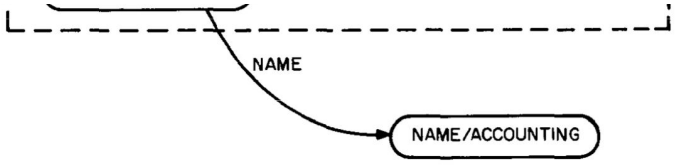


Entity Set Model

of role-name/entity-name-set-name/entity-name triplets shown in Figure 23. Conceptually, the entity set model differs from the entity-relationship model in the following ways:

(1) In the entity set model, everything is treated as an entity. For example, "COLOR/BLACK" and "NO–OF–YEARS/45" are entities. In the entity-relationship model, "blue" and "36" are usually treated as values. Note treating values as entities may cause semantic problems. For example, in Figure 22, what is the difference between "EMPLOYEE–NO/2566", "NAME/Peter Jones", and "NAME/Sam Jones"? Do they represent different entities?

(2) Only binary relationships are used in the entity set model,[5] while n-ary relationships may be used in the entity-relationship model.

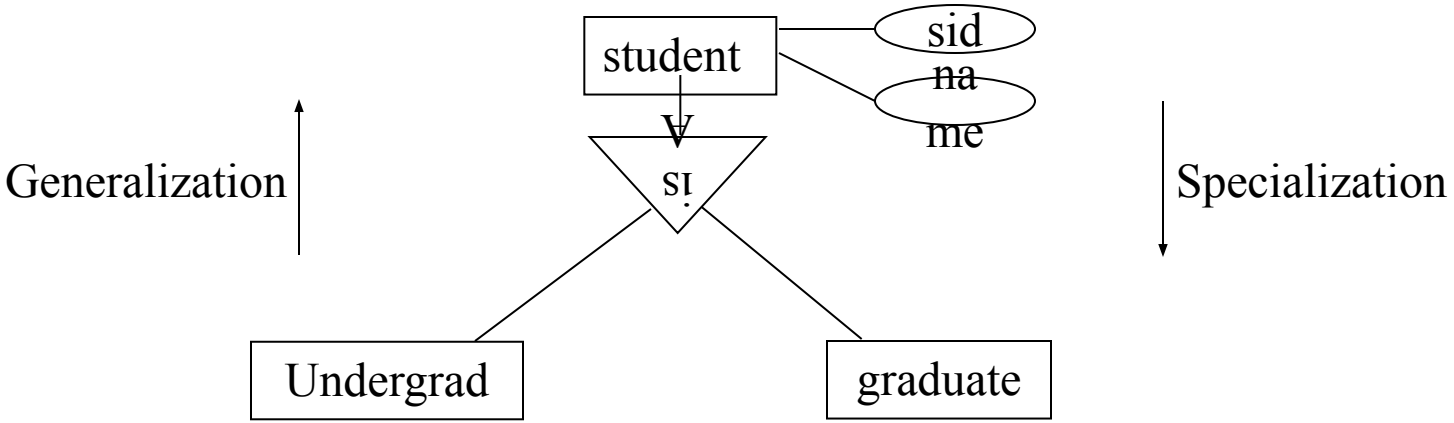


Discussion

GENERALIZATION AND SPECIALIZATION

- ***Generalization*** is the result of computing the union of two or more entity sets to produce a higher-level entity set. It represents the containment relationship that exists between the higher-level entity set and one or more lower-level entity sets.

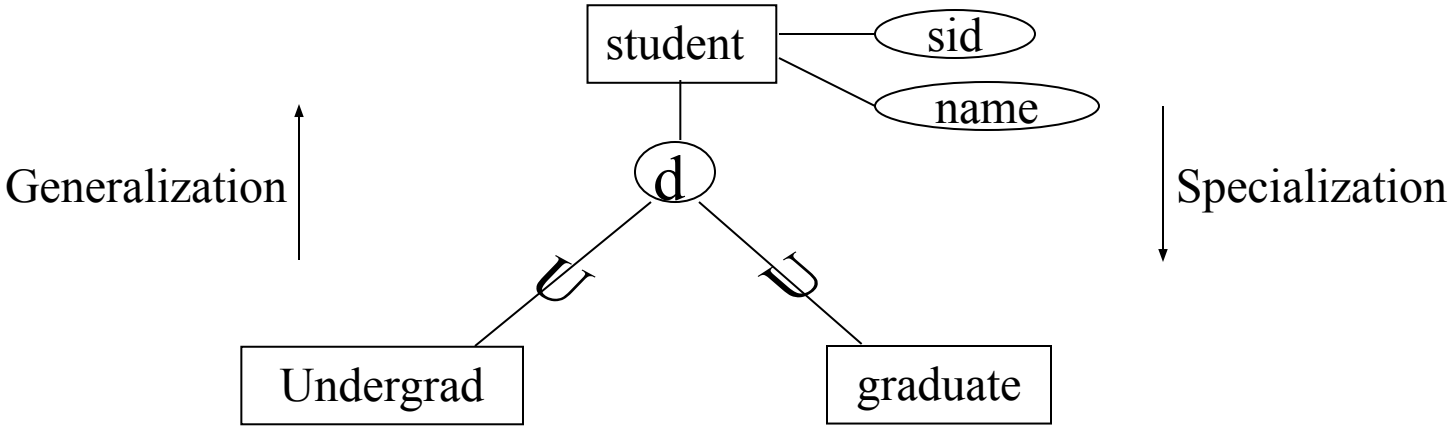
- ***Specialization*** constructs the lower level entity sets that are a subset of a higher level entity set.



*

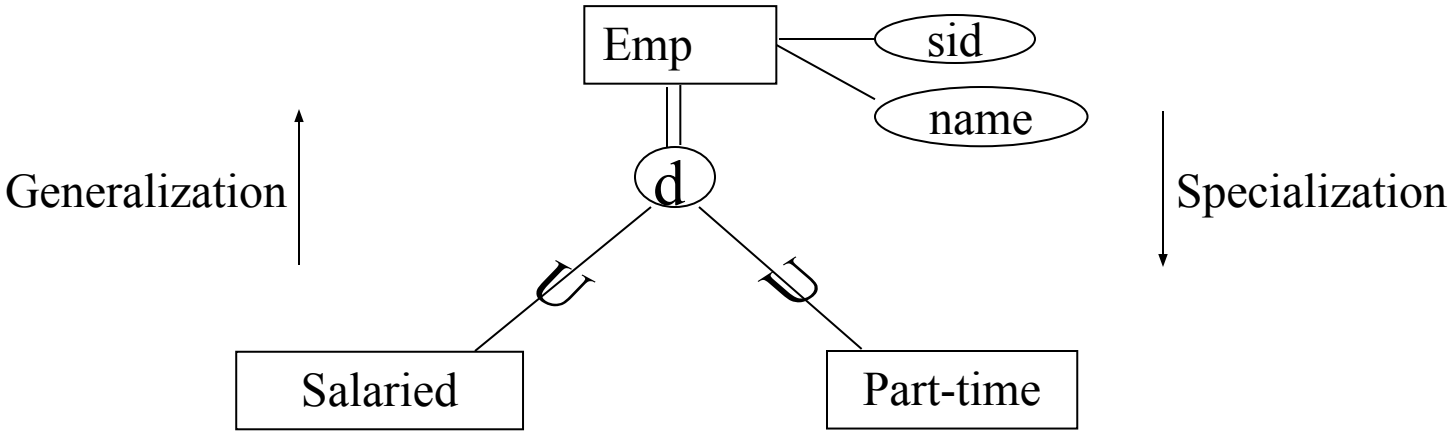
GENERALIZATION AND SPECIALIZATION

- Undergrad and graduate are termed subclasses of the superclass student.
- This is a superclass/subclass or simply class/subclass relationship.
- A member of a subclass MUST be a member of the superclass.
- An alternative notation is the Union symbol
- The circle with d specifies that the specializations are disjoint. A member of Undergrad entity set may NOT be a member of the graduate entity set.



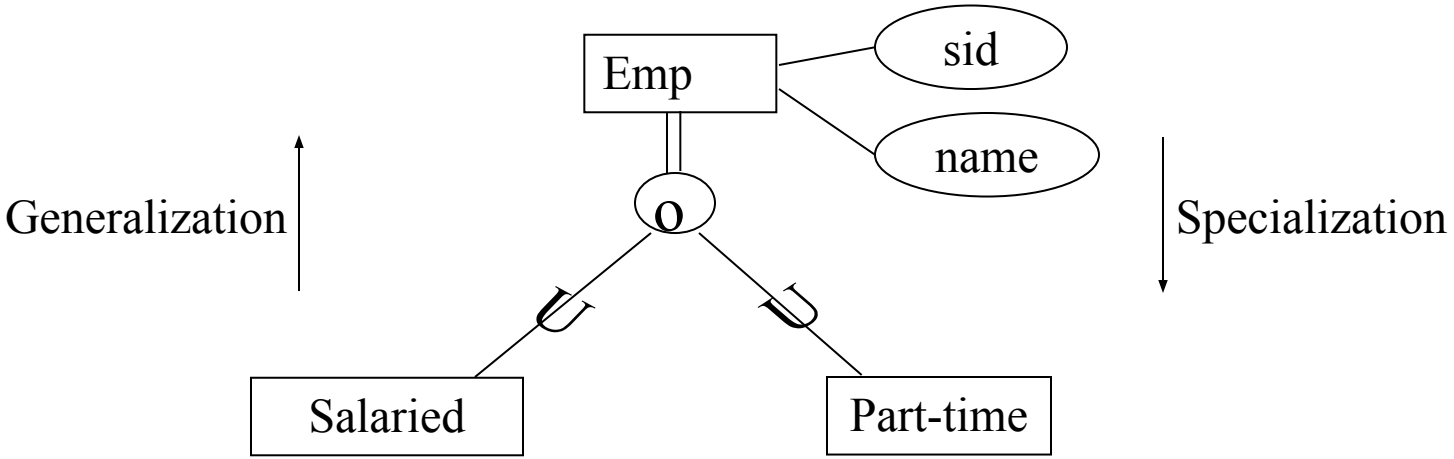
GENERALIZATION AND SPECIALIZATION

- A design may require all members of an entity-set to be specialized. For example, an employee MUST be a member of either a Salaried or Part-time. Use double lines to dictate this constraint.



GENERALIZATION AND SPECIALIZATION

- One may allow the specialized entity sets to overlap. For example, an entity might be both a Salaried and Part-time. "o" stands for Overlap when specializing.



*

Relational Data Model: Basic Concepts

- Relational data model consists of a collection of tables, each with a unique name. A table is termed a ***relation***.
- A **row** (also termed a **tuple** or a **record**) in table R represents a relationship among a set of values. Each value corresponds to an attribute or a field. A column of a table represents an attribute. Given a relation with attributes $(A_1{:}D_1, A_2{:}D_2, ..., A_n{:}D_n)$ the domain of this relation is $D_1 \times D_2 \times .... \times D_n$. However, an instance of a relation is a subset of this domain (recall "value & a variable"), e.g., CSCI485 (stid:integer, name:string, gpa:float)

stid	name	gpa
101	Adams	3.4
102	Jones	2.9
103	Hayes	3.5
104	Smith	3.1

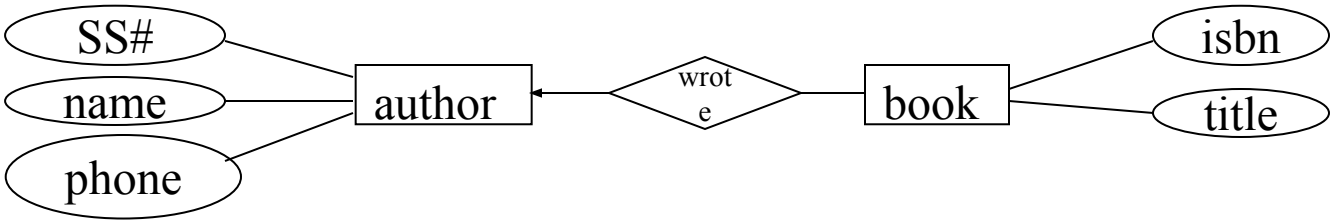
TABLES GENERATION

- Transforming an entity set to a table (relation):
  1. Strong entity set with attributes $a_1, a_2, ..., a_n$: represent it as a table with n unique columns (one column per attribute).
  2. Weak entity set with attributes $a_1, a_2, ..., a_n$ and a foreign key $b_1, b_2, ..., b_m$: represent it as a table with $n+m$ columns, one for each of $\{a_1, a_2, ..., a_n\} \cup \{b_1, b_2, ..., b_m\}$.
- Transforming many-to-many relationship sets to tables:
  1. Relationship set involving entity sets $E_1, E_2, ..., E_n$, each entity having primary key *prim-key*(E_i), represent it as a relation with each attribute corresponding to
     - $\bigcup_{i=1}^{n}$ **Prim_key(E_i)**

  1. if the relationship has attributes $a_1, a_2, ..., a_n$ then the relation consists of:
     - **$\{a_1, a_2, ..., a_n\} \cup \bigcup_{i=1}^{n}$ Prim_key (E_i)**

TABLES GENERATION (Cont…)

3. For a functional relationship (1:1, M:1, 1:M) from set A to set B, incorporate the primary key of B into A



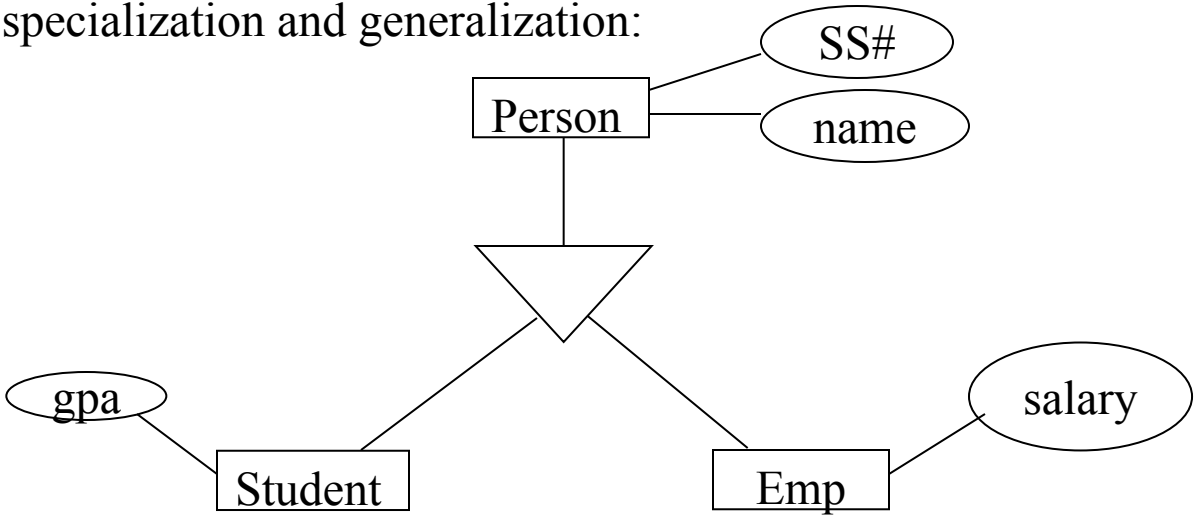
Author(SS#, name, phone)

Book(SS#, isbn, title)

- The number of attributes in a relation R is termed ***arity*** or ***degree*** of relation R.
- The number of tuples in a relation R is termed the ***cardinality*** of R.

TABLES GENERATION (Cont…)

- For specialization and generalization:



TABLES GENERATION (Cont…)

Two ways to generate tables:

1. Approach 1:
   - A table for the higher level entity set
   - For each lower entity set, create a table that includes a column for each of the attributes of that entity set plus a column for each attribute of the primary key of the higher-level entity set

     Person(SS#, name)

     Student(SS#, gpa)

     Emp(SS#, salary)
2. Approach 2:
   - No table for the higher level entity set
   - One table for each lower entity set with the following columns:
     - One column for each local attribute
     - One column for each attribute of the higher level set

     Student(SS#, name, gpa)

     Emp(SS#, name, salary)

*